

I. THE CORE CHALLENGE: HOW MACHINES "READ"

Computers are excellent at processing numbers but incapable of understanding the "meaning" of a string like "Apple." Historically, NLP used **One-Hot Encoding**, where each word was a vector of 0s with a single 1 at its unique index.

1.1. The Failure of One-Hot Encoding

- **Sparsity:** If your vocabulary has 50,000 words, every word is a 50,000-dimension vector. This is a massive waste of memory.
- **No Relationship:** In One-Hot space, the distance between "Hotel" and "Motel" is the same as the distance between "Hotel" and "Cat." The computer sees no similarity between related concepts.

Word Embeddings solve this by mapping words into a **Dense Vector Space** (usually 100 to 768 dimensions) where words with similar meanings are physically close to each other.

II. THE DISTRIBUTIONAL HYPOTHESIS

The magic of embeddings is based on the **Distributional Hypothesis**: *"A word is characterized by the company it keeps."* If "Coffee" and "Tea" both frequently appear near the words "drink," "cup," and "hot," the AI concludes they must be related, even if it doesn't know what a liquid is.

III. WORD2VEC: THE PIONEER ARCHITECTURE

Developed by Google in 2013, **Word2Vec** uses a shallow neural network to learn word associations. It offers two training methods:

3.1. Continuous Bag of Words (CBOW)

- **Goal:** Predict a "Target Word" based on its surrounding "Context Words."
- **Best for:** Small datasets; it is faster and represents frequent words well.

3.2. Skip-gram

- **Goal:** Use a "Target Word" to predict the "Context Words" around it.
- **Best for:** Large datasets; it is better at representing rare words.

3.3. The Famous Equation

The result of Word2Vec is a mathematical space where semantic relationships can be solved with simple vector arithmetic:

$$King - Man + Woman \approx Queen$$

IV. GLOVE: GLOBAL VECTORS FOR WORD REPRESENTATION

While Word2Vec looks at local windows of text, **GloVe** (Stanford, 2014) looks at the **Global Co-occurrence Matrix** of the entire dataset.

- **Logic:** If "Solid" and "Ice" appear together many times across millions of documents, their vectors should be highly correlated.
- **Advantage:** GloVe often captures global statistics better than Word2Vec, making it very stable for general-purpose NLP tasks.

V. TEXT PREPROCESSING: CLEANING THE DATA

Before text can be turned into embeddings, it must be "cleaned." This is a critical step in your RAG pipeline.

1. **Tokenization:** Breaking a sentence into individual words or "sub-words."
2. **Lowercasing:** Ensuring "Apple" and "apple" are treated as the same word.
3. **Stop-word Removal:** Removing common words like "the," "is," and "at" which carry little semantic meaning (though this is often skipped for deep learning models like BERT).
4. **Stemming/Lemmatization:** Reducing words to their root form (e.g., "running" $\rightarrow$ "run").

VI. PRACTICAL LAB: GENERATING EMBEDDINGS (GENSIM)

This Python snippet shows how to train a Word2Vec model on a custom dataset (like your 50 PDF files).

Python

```
from gensim.models import Word2Vec
```

```
# 1. Sample tokenized sentences from your project
```

```
sentences = [["data", "engineering", "is", "cool"],  
             ["machine", "learning", "uses", "data"],  
             ["python", "is", "the", "best", "for", "ml"]]
```

```
# 2. Train the model
```

```
# vector_size: 100 dimensions
```

```
# window: look at 5 words at a time
```

```
# min_count: ignore words that appear less than once
```

```
model = Word2Vec(sentences, vector_size=100, window=5, min_count=1, workers=4)
```

3. Find similar words

```
similar_words = model.wv.most_similar("data")  
  
print(f'Words similar to 'data': {similar_words})
```

4. Get the raw vector for a word

```
vector = model.wv['python']  
  
print(f'Vector for 'python' (first 5 dims): {vector[:5]})
```